BỘ BÀI TẬP THỰC HÀNH

LangChain Tool Calling và thuật toán

4 tình huống doanh nghiệp: đối soát hóa đơn, công suất sản xuất, lịch gửi thông báo và tác động sự cố.

01 Hai khoản chuyển tiền nào trả đủ hóa đơn?

02 Cần máy nhanh cỡ nào để kịp ca?

03 Gửi thông báo khi nào cho hợp lý?

04 Một dịch vụ hỏng sẽ ảnh hưởng đến đâu?

AI chọn công cụ. Backend tính toán. Người dùng kiểm tra kết quả.

Nội dung

Bốn bài tập có thể làm riêng hoặc ghép thành một web app. Mỗi bài cho thấy AI gọi công cụ nào và thuật toán giải quyết việc gì.

Bài 01 - Hai khoản chuyển tiền nào trả đủ hóa đơn? 3

Bài 02 - Cần máy nhanh cỡ nào để kịp ca sản xuất? 5

Bài 03 - Gửi thông báo khi nào để không làm phiền khách? 6

Bài 04 - Một dịch vụ hỏng sẽ ảnh hưởng đến đâu? 7

Giới thiệu và cách làm

Bạn sẽ xây gì?

Người dùng đặt câu hỏi bằng lời thường, ví dụ: “Hai khoản chuyển tiền nào trả đủ hóa đơn này?”. GenAI không có sẵn số liệu trong đầu. Nó phải **gọi công cụ** để đọc hóa đơn và giao dịch đang có trong hệ thống. Sau đó, một công cụ do bạn viết chạy thuật toán để tìm kết quả. GenAI giải thích kết quả bằng dữ liệu công cụ trả về.

Đó là điểm khác với bài Structured Output trước: ở đây trọng tâm là **chọn đúng công cụ, gọi đúng tham số và dùng kết quả công cụ**, không chỉ chuyển một đoạn văn thành JSON. Các hệ thống và số liệu trong đề đều là dữ liệu giả để luyện tập.

Bài	Câu hỏi từ người dùng	Thuật toán cần viết
01 - Đối soát hóa đơn	Hai khoản chuyển tiền nào cộng lại vừa đủ hóa đơn?	Hash map, tìm hai số có tổng cho trước
02 - Công suất sản xuất	Cần máy nhanh cỡ nào để xong việc trong ca?	Tìm kiếm nhị phân trên đáp án
03 - Lịch gửi thông báo	Gửi khi nào để không làm phiền cùng một khách quá dồn dập?	Hàng đợi ưu tiên và thời gian chờ
04 - Tác động sự cố	Một dịch vụ hỏng sẽ ảnh hưởng những phần nào?	Duyệt đồ thị theo chiều rộng (BFS)

Cách làm chung

1. Làm một web app có ô chat. Người dùng hỏi bằng tiếng Việt và xem được câu trả lời, dữ liệu được dùng, cùng lịch sử công cụ đã gọi.

2. Định nghĩa công cụ bằng LangChain **@tool**, gắn vào model bằng **bind_tools**. Khi model trả **tool_calls**, backend kiểm tra tên công cụ và tham số, tự chạy công cụ, rồi trả kết quả về model bằng **ToolMessage** có đúng **tool_call_id**. Lặp có giới hạn bước cho đến khi model trả câu trả lời cuối.
3. Công cụ đọc dữ liệu phải lấy từ database hoặc API giả lập của bài. **Không lấy số liệu do model tự điền** làm căn cứ tính toán. Phần thuật toán nằm trong Python/backend và có thể test mà không gọi model.
4. Công cụ chỉ được làm đúng việc đã khai báo. Không cho model gọi SQL tùy ý, URL tùy ý hoặc tên hàm ngoài danh sách. Nếu dữ liệu thiếu, công cụ lỗi hay không có đáp án, app nói rõ thay vì bịa kết quả.
5. Với hành động có thể thay đổi dữ liệu, chỉ tạo **đề xuất chờ người dùng duyệt**. Các bài này không yêu cầu tự chuyển tiền, gửi thông báo hay thay đổi hệ thống thật.
6. Có bộ dữ liệu giả và chế độ chạy thử không cần API key. Lưu câu hỏi, các lần gọi công cụ (tên, tham số, kết quả hoặc lỗi), kết quả thuật toán và câu trả lời để có thể kiểm tra lại. Giữ API key ở backend.

Nộp bài: source frontend/backend, hướng dẫn chạy, dữ liệu mẫu, test thuật toán cho ca bình thường và ca lỗi/biên, ảnh hoặc video luồng chat, và một đoạn ngắn giải thích vì sao GenAI cần gọi công cụ trong bài của bạn. Có thể làm từng bài riêng; không bắt buộc ghép bốn bài vào một app.

Chấm điểm (100): Tool Calling đúng luồng và an toàn 30; thuật toán và kết quả 30; web app và cách giải thích 15; test 15; dữ liệu mẫu và khả năng xem lại lịch sử 10.

Cơ sở thực tế

- Đối soát chuyển khoản với hóa đơn là một luồng có thật trong tài liệu Stripe.
- Lập kế hoạch theo tốc độ và công suất máy là một phần của quản lý work center trong SAP.
- Các nền tảng marketing áp dụng giới hạn tần suất gửi thông báo để tránh gửi quá nhiều cho cùng người nhận.
- Việc xem quan hệ giữa các dịch vụ khi có sự cố xuất hiện trong sơ đồ ứng dụng của AWS CloudWatch.
- Luồng **bind_tools**, **tool_calls** và **ToolMessage** được mô tả trong tài liệu LangChain.

Bài 01 - Hai khoản chuyển tiền nào trả đủ hóa đơn?

Một công ty có hóa đơn chưa thanh toán. Khách hàng nói đã chuyển tiền thành **hai lần**, nhưng nội dung chuyển khoản không ghi rõ mã hóa đơn. Kế toán phải mở hóa đơn, lọc các giao dịch chưa đối soát của đúng khách, rồi tìm hai khoản cộng lại vừa đủ số tiền còn thiếu.

Việc bạn cần làm: tạo trang chat để kế toán hỏi “Kiểm tra hóa đơn I-77”. AI gọi công cụ lấy số liệu, công cụ thuật toán tìm cặp giao dịch, rồi app hiển thị một **đề xuất đối soát** để kế toán xem. App không tự ghi nhận hóa đơn đã thanh toán.

AI cần gọi công cụ nào?

Công cụ	Dữ liệu trả về
get_open_invoice(invoice_id)	Mã khách, loại tiền, số tiền còn thiếu, trạng thái hóa đơn
list_unmatched_transfers(customer_id, currency)	Các khoản chuyển tiền chưa ghép với hóa đơn nào
find_two_transfer_match(invoice_id)	Kết quả thuật toán: cặp giao dịch, hoặc lý do không tìm được

Công cụ	Dữ liệu trả về
propose_invoice_match(invoice_id, transfer_ids)	Bản nháp chờ kế toán duyệt; không thay đổi sổ sách

AI phải dùng ID hóa đơn người dùng hỏi. Sau khi đọc hóa đơn, nó mới biết cần lấy giao dịch của **khách nào, loại tiền nào**. find_two_transfer_match phải tự đọc lại dữ liệu đã kiểm tra từ backend; không tin danh sách số tiền do model tự truyền vào.

Thuật toán cần viết

Chỉ xét giao dịch **unmatched**, cùng **customer_id**, cùng **currency**, số tiền dương. Sắp theo thời điểm nhận rồi mã giao dịch để kết quả ổn định. Dùng **hash map**: với mỗi khoản tiền **x**, kiểm tra xem đã gặp khoản **amount_due_minor - x** chưa. Nếu có nhiều cặp, chọn cặp xuất hiện sớm nhất theo thứ tự quét. Một giao dịch không được ghép với chính nó. Tiền dùng số nguyên theo đơn vị nhỏ nhất, không dùng số thực.

Dữ liệu mẫu: hóa đơn **I-77** của khách **C-12** còn thiếu **1.500.000 VND**.

Giao dịch	Giờ nhận	Khách	Loại tiền	Số tiền	Trạng thái
P1	09:00	C-12	VND	900.000	unmatched
P4	09:02	C-99	VND	600.000	unmatched
P5	09:03	C-12	USD	600.000	unmatched
P2	09:05	C-12	VND	600.000	unmatched
P3	09:10	C-12	VND	600.000	unmatched

Kết quả cần ra: P1 + P2 = 1.500.000 VND. Không chọn P4 vì sai khách, P5 vì sai loại tiền. P1 + P3 cũng đủ tiền nhưng P2 đến sớm hơn. Nếu không có cặp nào, trả **no_match** và giải thích, không tự chọn cặp “gần đúng”. Với **n** giao dịch hợp lệ, phần tìm cặp cần khoảng **O(n)** thời gian sau bước sắp xếp.

Trang web và kiểm tra

- Hiện câu hỏi, lịch sử các tool đã gọi, hóa đơn, hai giao dịch được đề xuất và lý do chọn.
- Thử hóa đơn không tồn tại, đã thanh toán, giao dịch đã được ghép, sai khách/loại tiền, hai giao dịch có cùng số tiền và trường hợp không có cặp.
- Nếu người dùng yêu cầu “hãy ghi nhận luôn”, app chỉ tạo đề xuất rồi đợi người có quyền duyệt.

Nếu muốn làm thêm: hỗ trợ một khoản thanh toán đủ tiền hoặc ba khoản cộng lại. Cần nêu rõ cách chọn khi có nhiều cách ghép.

Bài 02 - Cần máy nhanh cỡ nào để kịp ca sản xuất?

Trưởng ca hỏi: “Với các lô hàng còn lại, máy nào có thể xử lý xong trước khi hết ca?”. Số lượng mỗi lô nằm trong hệ thống sản xuất, còn số giờ làm và danh sách máy đang rảnh nằm ở hai nơi khác. AI cần gọi công cụ để gom đủ thông tin trước khi trả lời.

Việc bạn cần làm: web app nhận câu hỏi về một dây chuyền và một ca làm, cho thấy tốc độ xử lý tối thiểu cần có, máy nào đáp ứng và cách tính.

AI cần gọi công cụ nào?

Công cụ	Dữ liệu trả về
get_open_batches(line_id)	Các lô chưa làm và số sản phẩm của từng lô
get_shift_hours(line_id, shift_id)	Số giờ còn dùng được trong ca
get_available_machines(line_id)	Máy đang rảnh và tốc độ mỗi máy (sản phẩm/giờ)
calculate_min_required_rate(line_id, shift_id)	Tốc độ tối thiểu do thuật toán tính từ dữ liệu backend

Nếu người dùng chưa nói **line_id** hoặc **shift_id**, AI phải hỏi lại. Các công cụ đọc theo ID đã được kiểm tra; model không được tự bịa danh sách lô hoặc công suất máy. **calculate_min_required_rate** phải chạy bằng code, không nhờ model tính nhẩm.

Thuật toán cần viết

Trong bài này, **một máy** xử lý từng lô lần lượt. Một lô có **q** sản phẩm, máy làm được **r** sản phẩm/giờ thì lô đó chiếm **ceil(q / r)** giờ. Không ghép phần giờ lẻ của hai lô; thời gian đổi lô coi như bằng 0. Tổng giờ cần không được vượt số giờ còn lại.

Hãy tìm **tốc độ nguyên nhỏ nhất r** vẫn kịp ca. Nếu máy tốc độ **r** làm kịp, mọi máy nhanh hơn cũng làm kịp; vì vậy dùng **tìm kiếm nhị phân** thay vì thử từng tốc độ từ 1 trở lên. Nếu số giờ còn lại ít hơn số lô có sản phẩm, trả **impossible_under_rules** vì mỗi lô cần ít nhất một giờ.

Dữ liệu mẫu: bốn lô còn **30, 11, 23, 4** sản phẩm; ca còn **8** giờ. Máy rảnh: **M1=8**, **M2=12**, **M3=16** sản phẩm/giờ.

Kết quả cần ra: tốc độ tối thiểu là **11 sản phẩm/giờ**. Ở tốc độ 10, bốn lô cần **3 + 2 + 3 + 1 = 9** giờ; ở tốc độ 11 cần **3 + 1 + 3 + 1 = 8** giờ. Trong các máy đang rảnh, chọn **M2** vì đây là máy chậm nhất vẫn đạt yêu cầu. Nếu không máy nào đủ nhanh, báo thiếu công suất và không tự nhận ca đã khả thi.

Trang web và kiểm tra

- Hiện lô hàng, số giờ, tốc độ tối thiểu, máy được đề xuất và phép tính cho từng lô.
- Thử không có lô, lô có số lượng 0, giờ ca bằng 0, thiếu máy phù hợp, dữ liệu máy đã đổi sau khi AI đọc, và trường hợp chính xác bằng hạn ca.
- Nếu người dùng muốn “đặt máy M2”, app chỉ tạo đề xuất chờ trưởng ca xác nhận; không tự khóa máy.

Nếu muốn làm thêm: đưa thời gian chuẩn bị máy giữa hai lô vào phép tính. Khi đó phải giải thích rõ quy tắc thời gian mới trước khi chọn máy.

Bài 03 - Gửi thông báo khi nào để không làm phiền khách?

Đội marketing có nhiều thông báo đã chuẩn bị sẵn. Một khách có thể xuất hiện trong vài chiến dịch cùng lúc. Gửi dồn ba tin cho cùng người trong một buổi sáng dễ gây khó chịu. Nhân viên muốn hỏi app: “Hôm nay nên xếp các tin này vào khung giờ nào?”.

Việc bạn cần làm: tạo một lịch gửi **đề xuất**, vừa hoàn tất sớm vừa giữ khoảng cách tối thiểu giữa hai tin gửi cho cùng khách. Bài học dùng quy tắc theo ô thời gian để dễ kiểm tra; đây là mô phỏng đơn giản của việc giới hạn tần suất gửi trong doanh nghiệp.

AI cần gọi công cụ nào?

Công cụ	Dữ liệu trả về
`get_pending_messages(campaign_id)`	Tin đang chờ gửi, mã tin và mã người nhận
`get_contact_policy(campaign_id)`	Các khung giờ được dùng, số tin tối đa mỗi khung, số ô phải cách giữa hai tin cho cùng khách
`get_opt_in_status(customer_ids)`	Khách nào đã đồng ý nhận tin, khách nào chưa
`build_send_schedule(campaign_id)`	Lịch do thuật toán tạo từ dữ liệu mới nhất trong backend

AI đọc các công cụ để hiểu tình hình rồi gọi `build_send_schedule`. Công cụ xếp lịch phải kiểm tra quyền nhận tin lại trước khi tính; không tin một danh sách khách do model tự tạo. Không có công cụ gửi tin thật trong bài này.

Thuật toán cần viết

Mỗi ô thời gian gửi tối đa **một** tin. Sau khi gửi cho khách A tại ô số `i`, phải có ít nhất `gap=2` ô khác ở giữa trước khi gửi tin tiếp theo cho A; A có thể nhận tiếp sớm nhất ở ô `i+3`. Tin của khách chưa đồng ý nhận phải được loại ra trước khi xếp lịch.

Dùng **max-heap** để ưu tiên khách còn nhiều tin chưa gửi và một hàng đợi giữ khách đang phải chờ. Nếu không có khách nào đủ điều kiện ở một ô, đặt `idle` và chuyển sang ô tiếp theo. Khi cùng số tin còn lại, chọn mã khách theo alphabet; trong cùng một khách, chọn mã tin theo alphabet. App phải báo tin chưa xếp được nếu hết khung giờ, không tự đẩy sang ngày khác.

Dữ liệu mẫu: 8 ô từ 09:00 đến 16:00, mỗi ô dài một giờ. Cả A và B đã đồng ý nhận tin. A có `A1, A2, A3`; B có `B1, B2, B3`; `gap=2`.

Ô	09:00	10:00	11:00	12:00	13:00	14:00	15:00	16:00
Kết quả	A1	B1	idle	A2	B2	idle	A3	B3

Kết quả cần ra: 6 tin được xếp trong 8 ô như bảng. A nhận ở ô 0, 3, 6; B ở ô 1, 4, 7. Không có ai nhận hai tin quá gần nhau. AI trình bày đây là **lịch đề xuất**, chưa phải lịch đã gửi.

Trang web và kiểm tra

- Hiện công cụ đã gọi, quy tắc liên hệ, trạng thái đồng ý nhận, lịch từng ô và các ô **idle**.
- Thử khách từ chối nhận tin, chỉ có một khách, **gap=0**, nhiều khách cùng số tin, hết giờ nhưng còn tin và công cụ đọc dữ liệu bị lỗi.
- Nếu người dùng yêu cầu “gửi ngay”, app giải thích cần người có quyền duyệt và không gọi API gửi tin trong bài tập.

Nếu muốn làm thêm: thêm giờ yên lặng hoặc giới hạn số tin mỗi ngày. Cần kiểm tra lại toàn bộ lịch sau khi thêm quy tắc mới.

Bài 04 - Một dịch vụ hỏng sẽ ảnh hưởng đến đâu?

Một ứng dụng bán hàng gồm nhiều dịch vụ nhỏ. Khi cơ sở dữ liệu gặp sự cố, nhóm vận hành cần biết trang web, ứng dụng di động hay trang báo cáo có thể bị ảnh hưởng theo đường nào. Chỉ nhìn một cảnh báo lỗi thường chưa đủ để thấy toàn bộ chuỗi phụ thuộc.

Việc bạn cần làm: tạo trang chat để người trực vận hành hỏi về một sự cố. AI gọi công cụ đọc thông tin sự cố và sơ đồ dịch vụ. Công cụ thuật toán trả danh sách phần bị ảnh hưởng cùng **đường đi ngắn nhất** từ dịch vụ lỗi tới từng phần đó.

AI cần gọi công cụ nào?

Công cụ	Dữ liệu trả về
get_incident(incident_id)	Dịch vụ bị lỗi, thời điểm, trạng thái sự cố
get_service_graph(system_id)	Các quan hệ phụ thuộc và phiên bản sơ đồ
get_customer_facing_services(system_id)	Những dịch vụ người dùng trực tiếp thấy
trace_impact(incident_id)	Kết quả duyệt đồ thị từ dữ liệu backend

AI không được đoán đường ảnh hưởng từ tên dịch vụ. **trace_impact** đọc lại sơ đồ và sự cố theo ID để tránh dùng dữ liệu cũ hoặc cạnh do model bịa. Chỉ phân tích; bài này không tự khởi động lại dịch vụ.

Thuật toán cần viết

Xem mỗi dịch vụ là một điểm. Mũi tên **A → B** trong dữ liệu bài này nghĩa là **B phụ thuộc vào A**: nếu A lỗi, B có thể bị ảnh hưởng. Bắt đầu từ dịch vụ lỗi, dùng **BFS** để đi từng lớp: dịch vụ trực tiếp phụ thuộc, rồi dịch vụ phụ thuộc gián tiếp. Ghi nhớ điểm đã đi qua để không lặp vô hạn nếu sơ đồ có vòng. Lưu “điểm trước đó” để dựng lại đường đi ngắn nhất đến từng dịch vụ người dùng thấy.

Dữ liệu mẫu: sự cố **INC-9** tại **DB** của hệ thống **SHOP**; sơ đồ phiên bản **v7**:

```
DB → Auth      DB → Reports
Auth → Checkout Auth → MobileApp
Checkout → WebStore
```

Các dịch vụ người dùng thấy là **Reports**, **MobileApp**, **WebStore**. **Kết quả cần ra:**

Dịch vụ	Đường ảnh hưởng ngắn nhất	Số cạnh
Reports	DB → Reports	1
MobileApp	DB → Auth → MobileApp	2
WebStore	DB → Auth → Checkout → WebStore	3

Xếp kết quả theo số cạnh tăng dần, rồi mã dịch vụ theo alphabet. Nói rõ đây là **phần có thể bị ảnh hưởng theo sơ đồ phụ thuộc**, không khẳng định cả ba đã thực sự hỏng. Nếu một dịch vụ không có đường đi từ DB, không đưa vào danh sách ảnh hưởng. Với **V** dịch vụ và **E** quan hệ, BFS cần **O(V + E)** thời gian.

Trang web và kiểm tra

- Hiện trạng thái sự cố, phiên bản sơ đồ, danh sách dịch vụ có thể bị ảnh hưởng và đường đi tới từng dịch vụ.
- Thử ID sự cố không tồn tại, cạnh trùng, vòng trong sơ đồ, dịch vụ không nối tới DB, sơ đồ thiếu node và sơ đồ đã đổi phiên bản trong lúc xử lý.
- Nếu dữ liệu sơ đồ thiếu hoặc quá cũ, trả cảnh báo **incomplete_data**; đừng trình bày danh sách ảnh hưởng như chắc chắn đầy đủ.

Nếu muốn làm thêm: kết hợp số người dùng của từng dịch vụ để sắp thứ tự kiểm tra, nhưng vẫn phải phân biệt “có thể bị ảnh hưởng” với “đã xác nhận bị lỗi”.